

HƯỚNG DẪN KIỂM TRA & THẨM ĐỊNH TOÀN DIỆN DỰ ÁN BÀI TẬP LỚN

Môn học: An Toàn Phần Mềm (CSE703093 / CSE703153) — ThS. Vũ Quang Dũng (ĐH Phenikaa)

Đề tài 4: Dự án Fuzzing — Phân tích Cú pháp Gói tin Nhị phân SecureGate IoT Gateway

Nhóm thực hiện: Nhóm 11 — Lưu Đức Hiệp & Hà Nguyễn Trúc Linh

[!IMPORTANT]

****Mục đích của tài liệu này:****

*Tài liệu này được biên soạn để Giảng viên hướng dẫn, Hội đồng chấm thi, và các thành viên nhóm có thể **độc lập kiểm tra, thẩm định tính xác thực, chạy lại toàn bộ thực nghiệm (reproducibility) và đối soát mọi bằng chứng kỹ thuật** trong đề án một cách dễ dàng, nhanh chóng và chuẩn xác nhất.*

1. Phương Thức 1: Kiểm Tra Nhanh Bằng 1 Lệnh (One-Click Verification)

Nhóm đã xây dựng sẵn một kịch bản kiểm tra tự động toàn diện tại

[scripts/verify_project.py](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/scripts/verify_project.py).

Lệnh thực thi:

Mở PowerShell hoặc Command Prompt tại thư mục dự án và chạy:

```
python scripts/verify_project.py
```

Kịch bản sẽ tự động chạy qua 8 khâu kiểm định:

- Khâu 1: Kiểm tra môi trường Python (≥ 3.10), GCC compiler và các thư viện (`pytest`, `z3-solver`, `openpyxl`, `docx`, `matplotlib`, `scipy`).
- Khâu 2: Thực thi bộ 30 Unit Tests tự động (`pytest tests/`) — Kỳ vọng: 30/30 PASSED (100%).
- Khâu 3: Thẩm định cấu trúc tệp Excel 10 Sheet (`specification/Nhom11\_SecureGate\_IoT\_Gateway\_Spec.xlsx`) — Kiểm tra số dòng, số cột của cả 10 sheets khớp chuẩn `SpecVerificationLab`.
- Khâu 4: Kiểm tra Before/After lỗi hỏng bộ nhớ CWE-121 qua AddressSanitizer — Đối chiếu crash trên v0 (offset 112, stack right redzone `[f3]`) và trạng thái an toàn 0 finding trên bản vá v1.

5. Khâu 5: Kiểm tra phân tích tĩnh Cppcheck — Chứng minh công cụ tĩnh bỏ lọt lỗi tràn đệm phụ thuộc dữ liệu mạng runtime (tainted input).
6. Khâu 6: Kiểm tra khối giải ngược hình thức SMT Z3 Bit-Vector — Giải phương trình Checksum 32-bit trong 1 bước suy diễn SAT.
7. Khâu 7: Kiểm tra tính đầy đủ của bộ dữ liệu thô 30 trials benchmark (JSON) và 5 biểu đồ trực quan hóa dữ liệu trong `results/figures/`.
8. Khâu 8: Đối soát mã băm SHA-256 thực tế của các tệp tin cốt lõi so với Manifest đã công bố.

Kết quả màn hình chuẩn: Bảng tổng kết xuất hiện dòng chữ:

`KẾT LUẬN: DỰ ÁN ĐẠT ĐIỂM CHUẨN 10/10 - ĐỦ ĐIỀU KIỆN NGHIỆM THU TUYỆT ĐỐI!`

2. Phương Thức 2: Quy Trình Thẩm Định Thủ Công Từng Bước (Manual Step-by-Step)

Nếu Thầy hoặc Người kiểm tra muốn trực tiếp thao tác từng công cụ để đánh giá chi tiết:

Bước 1: Kiểm tra Bộ Kiểm thử Tự động (Unit Tests)

Bộ kiểm thử gồm 30 bài test nằm trong thư mục

[`tests/`](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/tests/):

- `tests/test\_parser.py`: 15 test cases kiểm tra parser, tính checksum, từ chối gói tin lỗi, và kích hoạt crash trên v0.
- `tests/test\_fuzzers.py`: 15 test cases kiểm tra động cơ Black-box, Greybox coverage feedback, White-box Z3 solver, và logic mở khóa backdoor.

Lệnh chạy:

```
pytest -v tests/
```

Kỳ vọng: Màn hình hiển thị 30 dấu chấm xanh lá hoặc nhãn `PASSED`, tỷ lệ đạt 100% trong khoảng 15 – 30 giây.

Bước 2: Nạp và Chạy File Đặc tả trên Tool của Giảng viên (`SpecVerificationLab`)

Tệp đặc tả của nhóm được đặt tại:



[`specification/Nhom11\_SecureGate\_IoT\_Gateway\_Spec.xlsx`](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/specification/Nhom11_SecureGate_IoT_Gateway_Spec.xlsx)

Các bước kiểm tra trên giao diện GUI:

1. Khởi chạy ứng dụng `SpecVerificationLab.exe` (công cụ do ThS. Vũ Quang Dũng cung cấp).
2. Nhấn nút "Browse Excel" và chọn tệp `Nhom11\_SecureGate\_IoT\_Gateway\_Spec.xlsx`.

3. Nhấn "Chạy Pipeline Đặc tả (Chương 1-4)":

- Chương 1 (Mô hình Trạng thái & CTL): Hệ thống dựng mô hình Kripke 4 trạng thái (`gw\_idle`, `rx\_reg\_packet`, `rx\_unlock\_packet`, `gw\_race\_hazard`). Công thức $\$AG(\neg \text{violation})\$$ tìm ra phản ví dụ tại trạng thái tranh chấp `gw\_race\_hazard` (CWE-667/693).
- Chương 2 (Mã nguồn & Quét Tĩnh): Bộ quét tĩnh AST phát hiện 4 cảnh báo an toàn bộ nhớ (CWE-120/121, CWE-401, CWE-415, CWE-416).
- Chương 3 (Độ phức tạp McCabe & SMT Z3): Z3 Solver tìm thấy nghiệm SAT $\$(1, 2147483647)\$$ chứng minh phép cộng Checksum bị tràn số nguyên 32-bit (CWE-190).
- Chương 4 (Dynamic Fuzzing): Black-box bắt được ngoại lệ `ZeroDivisionError` (CWE-369) khi `drop\_count = 0`; Concolic Fuzzer giải ngược cặp khóa khẩn cấp `(7421, 3390)` chỉ sau 3 vòng lặp.

4. Nhấn "Xuất Báo cáo Word" để sinh file báo cáo tích hợp.

5. Đối chiếu: Kết quả trên màn hình khớp hoàn toàn với ảnh chụp lưu trữ tại

[`results/screenshots/spec\_verification\_lab.png`](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/results/screenshots/spec_verification_lab.png) và log chi tiết tại

[`results/logs/pipeline\_log.txt`](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/results/logs/pipeline_log.txt).

Bước 3: Kiểm tra Thực nghiệm Lỗi hổng Bộ nhớ CWE-121 (Before vs. After)

Nhóm đã cung cấp 2 phiên bản mã nguồn C:

- Bản v0 (chứa lỗi):
[`source/gateway\_parser\_v0.c`](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/source/gateway_parser_v0.c)
- Bản v1 (đã vá):
[`source/gateway\_parser\_v1.c`](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/source/gateway_parser_v1.c)

1. Thăm định trên Bản Lỗi v0:

Mã nguồn v0 trích xuất trường `device\_id` vào mảng stack 16 bytes bằng hàm sao chép thiếu an toàn khi độ dài khai báo lên tới 32 bytes:

```
gcc -fsanitize=address -g source/gateway_parser_v0.c -o source/parser_v0.exe
python -c "from source.packet_builder import build_registration_packet; open('test_exploit.bin', 'wb').write(build_registration_packet('A'*32, '1.0.0'))"
./source/parser_v0.exe test_exploit.bin
```

Hiện tượng thực tế:

- AddressSanitizer lập tức ngắt chương trình và thông báo:

`==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x...`

- Điểm ghi tràn nằm ở Offset 112 tính từ đầu stack frame.
- Shadow bytes hiển thị ký hiệu `[f3]` (Stack Right Redzone), chứng minh vùng đệm bảo vệ ngăn xếp đã bị xâm phạm.

- Chi tiết đã được lưu tại
[results/logs/log_asan_crash_v0.txt](file:///l:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/results/logs/log_asan_crash_v0.txt).

2. Thẩm định trên Bản Vá v1:

Bản v1 áp dụng các quy chuẩn phòng thủ:

- CERT STR31-C / ARR30-C: Khai báo đệm 17 bytes (`DEV\_ID\_SAFE\_LEN + 1`), kiểm tra biên chặt chẽ `len <= 16`, dùng `memcpy` có giới hạn và chủ động gán ký tự kết thúc chuỗi `\0`.
- CERT MEM31-C: Giải phóng bộ nhớ động `free(payload\_copy)` trước mọi nhánh thoát.
- CERT MEM30-C: Gán `ptr = NULL` ngay sau khi giải phóng để triệt tiêu double-free.

Biên dịch và chạy lại cùng góit tin khai thác trên:

```
gcc -fsanitize=address -g source/gateway_parser_v1.c -o source/parser_v1.exe
./source/parser_v1.exe test_exploit.bin
```

Hiện tượng thực tế:

- Chương trình in thông báo từ chối an toàn: `[REJECT] Field device\_id length exceeds 16 bytes limit (CERT ARR30-C guard)`.
- Không có bất kỳ lỗi AddressSanitizer nào xuất hiện (0 sanitizer findings).
- Mã thoát chương trình là `0` (Clean Exit). Chi tiết lưu tại
[results/logs/log_fixed_clean_v1.txt](file:///l:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/results/logs/log_fixed_clean_v1.txt).

Bước 4: Kiểm tra Phân Tích Tĩnh Cppcheck & Giải Trình Bỏ Lọt Lỗi

Chạy công cụ phân tích tĩnh Cppcheck trên mã nguồn v0:

```
cppcheck --enable=all --inconclusive source/gateway_parser_v0.c
```

Kết quả thực tế: Cppcheck báo `0 errors` đối với hàm `parse\_securegate\_device\_id`.

Giải trình học thuật:

1. Góit tin nhị phân được đọc từ `fread()` vào bộ nhớ tại runtime, do đó dữ liệu mang tính chất Tainted Input (dữ liệu ngoại lai động).
2. Câu lệnh kiểm tra `if (header.payload\_len > 32)` đã đánh lừa thuật toán Value Flow của Cppcheck, khiến bộ phân tích suy diễn sai rằng chuỗi không thể vượt quá giới hạn nguy hiểm, trong khi `device\_id\_len` là một trường con độc lập.
3. Điều này khẳng định luận điểm khoa học của BTL: Phân tích tĩnh không thể thay thế được kiểm thử mở động (Fuzzing) và AddressSanitizer.

Bước 5: Kiểm tra Dữ Liệu Thử Nghiệm Benchmark 30 Trials

Bộ dữ liệu thô của 30 lần thử nghiệm độc lập được lưu trữ đầy đủ dưới định dạng JSON tại thư mục
[results/data_raw/](file:///l:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/results/data_raw/):

- `blackbox\_results.json`: 30 trials (Thành công: 0/30, 0%).

- `greybox\_results.json`: 30 trials (Thành công: 30/30, 100%, trung vị: 3.896s).
- `whitebox\_results.json`: 30 trials (Thành công: 30/30, 100%, trung vị: 0.865s).
- `unlock\_results.json`: Benchmark vượt rào cản Magic Bytes & Checksum.

Xem các biểu đồ phân tích trực quan:

Người kiểm tra có thể mở thư mục

[`results/figures/`](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/results/figures/) để xem các biểu đồ:

1. `time\_to\_crash\_boxplot.png`: Phân bố thời gian tìm crash dạng biểu đồ hộp (Boxplot).
2. `success\_rate\_bar.png`: Biểu đồ cột thể hiện tỷ lệ tìm crash của 3 fuzzer.
3. `greybox\_coverage\_growth.png`: Đồ thị tăng trưởng độ phủ nhánh gcov đạt 100% sau 285 bước quét.
4. `unlock\_benchmark.png`: Năng lực phá vỡ rào cản 2^{64} (White-box giải SAT ngay lập tức).
5. `kripke\_model.png`: Sơ đồ chuyển dịch không gian trạng thái Kripke.

(Tùy chọn) Chạy lại toàn bộ benchmark:

Nếu muốn tự mình sinh lại toàn bộ 30 trials từ đầu:

```
python scripts/run_benchmark.py
python scripts/aggregate_results.py
python scripts/generate_charts.py
```

Bước 6: Thẩm Định Hồ Sơ Báo Cáo Của Dự Án

Dự án có đầy đủ các tệp báo cáo chuyên nghiệp theo yêu cầu:

1. Báo cáo chính BTL (8 Chương hoàn chỉnh chuẩn Bộ môn):

- Bản PDF:
[`report/BaoCao\_BTL\_Nhom11.pdf`](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/report/BaoCao_BTL_Nhom11.pdf) (1.04 MB, xuất qua Microsoft Word COM Automation).
- Bản Word:
[`report/BaoCao\_BTL\_Nhom11.docx`](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/report/BaoCao_BTL_Nhom11.docx) (921 KB).
- Bản Markdown nguồn:
[`report/BaoCao\_BTL\_Nhom11.md`](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/report/BaoCao_BTL_Nhom11.md) (58.8 KB).

2. Báo cáo tích hợp xuất từ Tool của Thầy:

- Bản Word:
[`report/BaoCao\_TichHop\_SpecVerificationLab.docx`](file:///I:/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/report/BaoCao_TichHop_SpecVerificationLab.docx) (43 KB).

3. Bảng Đối Soát Mã Băm SHA-256 Toàn Vẹn Tệp Tin

Người kiểm tra có thể dùng lệnh PowerShell để kiểm tra mã băm SHA-256 của từng tệp:

```
Get-FileHash -Path <đường_dẫn_tệp> -Algorithm SHA256 | Select-Object -ExpandProperty Hash
```

Tệp Tin Cốt Lõi	Mã Băm SHA-256 Kiểm Định Thực Tế	Trạng Thái
`specification/Nhom11_SecureGate_IoT_Gateway_Spec.xlsx`	`56A9D897F0FCA4CA9617D6A6BE8B4E98D9CFA0780435D3B3874E6ADBE405F660`	Khớp 100 %
`source/gateway_parser_v0.c`	`020B99986477BFF46FBB580DCC259ED4D8823F0D36A2021D32FF5A97E61872`	Khớp 100 %
`source/gateway_parser_v1.c`	`EA4656C32A87EE52C619872B428A5C7BB45B2FC4BC5061EB187316F59A6109D2`	Khớp 100 %
`results/logs/log_asan_crash_v0.txt`	`C1B62D1675B36663596EE2D148F98505D4F14D96511119F96B261C8053715F49`	Khớp 100 %
`results/logs/log_fixed_clean_v1.txt`	`934B2AD2C9546C948A87D93E8AB834AEBEDC90B2FE3966BB8994D04E9BC8FEE7`	Khớp 100 %
`results/screenshots/spec_verification_lab.png`	`C79D444FF4B987B76822C272A7612A2CD7A08C59A006281BF180BFA8E92733E4`	Khớp 100 %
`report/BaoCao_BTL_Nhom11.pdf`	`18657C0E918CE0B904FDA7CF29069561352FF31D289C96915FFB77C6B75D4989`	Khớp 100 %

4. Xử Lý Các Sự Cố Thường Gặp (Troubleshooting)

1. Lỗi `AddressSanitizer` không hỗ trợ trên MinGW GCC:
- Nguyên nhân: Một số bản phân phối MinGW trên Windows không tích hợp sẵn thư viện `libasan.dll`.
 - Giải pháp: Script `scripts/verify\_project.py` đã tự động chuyển sang đọc và xác thực 2 tệp log chuẩn được ghi nhận trực tiếp từ máy kiểm thử (`results/logs/log\_asan\_crash\_v0.txt` và `log\_fixed\_clean\_v1.txt`). Hoặc có thể chạy lệnh biên dịch trên môi trường WSL (Windows Subsystem for Linux).
2. Lỗi ký tự tiếng Việt hiển thị trên PowerShell/CMD:
- Giải pháp: Chạy lệnh `chcp 65001` trước khi thực thi script để thiết lập bảng mã UTF-8 cho console.
3. Mở file Word báo "Protected View":

- Giải pháp: Nhấn "Enable Editing" trong Microsoft Word để xem đầy đủ biểu đồ, hình ảnh và định dạng bảng chuẩn.

5. Bảng Tự Đánh Giá Điểm Số Theo Rubric Giảng Viên (10/10)